

SmartSignage Player-AD

Manual completo

Instalação, configuração, operação, arquitetura e troubleshooting

Data: 2026-07-16

Versão de referência do app: 1.67

Índice

- 1. Resumo executivo
- 2. Manual do utilizador — instalação e configuração
- 3. Manual operacional (TV box / totem)
- 4. Notas técnicas de instalação (kit pendrive / ADB)
- 5. Manual de arquitetura e desenvolvimento
- 6. Troubleshooting — crash
- 7. Registro operacional

Documento gerado automaticamente a partir dos ficheiros Markdown do repositório.

1. Resumo executivo

Fonte: RESUMO-EXECUTIVO-PLAYER-AD.md

Player-AD — Resumo Executivo

Data: 2026-04-07

Objetivo do sistema

O Player-AD é um player Android TV para exibição de campanhas de mídia com alta disponibilidade operacional, mesmo em cenários de rede instável.

Capacidades-chave

- Recebe plano de mídia online (heartbeat + dispatch).
- Faz cache local de conteúdo para reduzir dependência de rede.
- Reproduz vídeo, imagem e **HTML** (WebView — cardápio ao vivo e animações Publicar em Tela) em loop contínuo.
- Opera em fallback offline com:
 - último plano persistido;
 - playlist local de propagandas e vinhetas.
- Suporte operacional com tela de debug, logs e estado offline.

Arquitetura em alto nível

- **UI/Orquestração:** MainActivity, DebugConfigActivity.
- **Playback:** PlayerController + Media3 (ExoPlayer).
- **Integração API:** DispatcherApiClient, PlayerEventsClient.
- **Persistência local:** MediaCacheManager, AppDirs, arquivos de estado/log.
- **Boot/Kiosk:** BootCompletedReceiver, modo imersivo fullscreen.

Valor para operação

- Continuidade de reprodução mesmo sem backend disponível.
- Diagnóstico rápido em campo (debug UI + log operacional).
- Instalação flexível:
 - via ADB (suporte técnico);
 - via pendrive (operação local sem notebook).

Riscos e controles

- **Conflito de assinatura APK:** mitigado por fluxo de desinstalação/reinstalação.

- **Mudanças de contrato backend:** concentradas em clientes API e parser de dispatch.
- **Problemas de storage/dispositivo:** mitigados por fallback de diretório (`externalFilesDir -> filesDir`).

Indicadores recomendados (governança)

- taxa de sucesso de heartbeat e dispatch;
- percentual de tempo em ONLINE VS PERSISTED VS FALLBACK_LOCAL;
- incidência de crash por versão;
- tempo médio de atualização em campo.

Próximos passos sugeridos

- padronizar versionamento de release (`versionCode/versionName`);
- incluir rotina automática de validação pós-instalação;
- consolidar dashboard simples de saúde do player por dispositivo.

2. Manual do utilizador — instalação e configuração

Fonte: MANUAL-USUARIO-INSTALACAO-CONFIGURACAO.md

Player-AD — Manual do utilizador

Instalação e configuração do player Android (TV box / totem digital).

Versão de referência do app: 1.67

Público: operador de campo, técnico de instalação e administrador do painel web

Documentação técnica (boot, kiosk avançado, logs): MANUAL-OPERACIONAL-TVBOX.md

1. O que é o Player-AD

O **Player-AD** é a aplicação Android que corre no totem (ou TV box) e:

- Liga-se ao servidor **Totem Digital / Smart Signage**
- Identifica-se com o **código de ativação (UIN)** do totem
- Descarrega e reproduz as mídias da playlist
- Envia heartbeat (estado online) ao servidor

Sem `serverUrl` + `uin` válidos, o player não consegue receber conteúdo.

2. O que precisa antes de começar

Item	Observação
Aparelho Android	TV box / totem com Android 7 ou superior
Rede	Wi-Fi ou cabo; o aparelho deve alcançar o servidor (IP ou DNS)
APK	Ficheiro <code>Player-AD-release.apk</code> (release assinado)
Código de ativação	Gerado no painel web ao criar/consultar o totem (campo Ativação / UIN)
URL do servidor	<code>http://217.216.91.135:8080</code> (sem barra no fim)

Onde obter o código de ativação (UIN)

- Abra o painel web do Totem Digital / Smart Signage
- Entre em **Totens** (ou equivalente)
- Crie o totem ou abra um existente
- Copie o código mostrado como **Ativação** (é o UIN)

Guarde esse código: será o mesmo valor em `uin` e, em geral, também em `deviceId` no player.

3. Instalar o aplicativo

Há três formas comuns. Use a que se adequar ao local.

3.1 Com pendrive (sem computador) — recomendado em campo

- Copie para o pendrive a pasta do kit (conteúdo de `install-pendrive/`), incluindo o APK em `apk/Player-AD-release.apk`
- Ligue o totem / TV box
- Insira o pendrive na porta USB
- Abra o **gestor de ficheiros**
- Entre em `apk/` e toque em `Player-AD-release.apk`
- Toque em **Instalar**
- Se o Android bloquear apps fora da loja:
 - no aviso, abra **Definições**
 - ative **Permitir desta fonte / Instalar apps desconhecidos**
 - volte ao APK e instale de novo
- Quando terminar, toque em **Abrir** (ou abra **SmartSignage Player-AD** no menu de apps)

3.2 Com PC e cabo USB (ADB) — kit pendrive

No PC, com Android Platform Tools (adb no PATH):

- No Android: ative **Opções de programador** → **Depuração USB**
- Ligue o cabo; aceite a autorização RSA no ecrã
- Na pasta `install-pendrive`, execute:

Windows (Prompt de Comandos):

```
scripts\install-from-pc-adb.bat
```

Linux / macOS / Git Bash:

```
bash scripts/install-from-pc-adb.sh
```

Opcional — enviar também a configuração:

```
bash scripts/install-from-pc-adb.sh "" config/exemplo-player-config.json
```

3.3 Com PC a partir do código-fonte (equipa técnica)

No repositório do projeto:

```
cd Player-AD\scripts
.\install-player-adb.ps1
```

Flags úteis:

Flag	Quando usar
-SkipBuild	Já tem o APK compilado
-NoKioskSetup	Celular / teste (não altera launcher nem rotação do sistema)
-OpenConfig	Abre a tela de configuração após instalar
-ConfigJson "C:\caminho\config.json"	Envia um JSON específico

APK gerado pelo build:

Player-AD\build\outputs\apk\release\Player-AD-release.apk

4. Configurar o player

Pode configurar **pelo ecrã do aparelho** (mais simples) ou com ficheiro **JSON** (útil em instalação em massa).

4.1 Abrir a tela de configuração no aparelho

- Abra o **Player-AD**
- Dê **5 toques rápidos** no centro do ecrã (ou no botão OK / centro do comando)
- Aparece um aviso do tipo “Mais N toque(s)...”
- Na quarta/quinta vez abre a **tela de configuração**

4.2 Campos obrigatórios

Preencha pelo menos:

Campo	O que colocar	Exemplo
URL do servidor (serverUrl)	Endereço do backend, sem barra no fim	http://217.216.91.135:8080
UIN / ativação (uin)	Código copiado do painel web	Tv-001 ou código gerado
ID do dispositivo (deviceId)	Em geral o mesmo que o UIN	Tv-001

Depois:

- Guarde / confirme a configuração
- Toque em **Aplicar e iniciar** (barra no rodapé)
- O player deve ligar ao servidor e começar a sincronizar a playlist

4.3 Orientação do ecrã (totem em pé)

Para totem **retrato (portrait)**:

Campo	Valor típico
screenOrientation	portrait
displayRotation	0 (retrato)

Valores de displayRotation:

Valor	Significado
0	Retrato
1	Paisagem
2	Retrato invertido
3	Paisagem invertida

Em alguns hardware Allwinner o sistema operativo também usa rotação (`user_rotation=1`); o script técnico de instalação trata disso. Se a imagem ficar deitada, contacte o suporte ou use o manual operacional.

4.4 Modo quiosque (kiosk)

Valor	Comportamento
strong	Ecrã cheio + bloqueio reforçado (totem em produção)
immersive	Só ecrã cheio imersivo (melhor para testes / celular)

Na tela de configuração o kiosk fica temporariamente relaxado para permitir editar.

4.5 Configuração por ficheiro JSON (opcional)

Local no aparelho:

- `/sdcard/smartsignage/player-config.json`

Modelo no kit:

- `install-pendrive/config/exemplo-player-config.json`

Exemplo mínimo:

```
{
  "serverUrl": "http://217.216.91.135:8080",
  "uin": "CODIGO-DE-ATIVACAO",
  "deviceId": "CODIGO-DE-ATIVACAO",
  "kioskMode": "strong",
  "displayRotation": 0,
  "screenOrientation": "portrait",
  "acceptImagesInPlaylist": true,
  "allowPlaybackAudio": true,
  "fallbackPropagandasPerVinheta": 3,
  "batimentoCardiaco": 120,
  "maxSecondsWithoutServerCheck": 600
}
```

Como colocar o ficheiro:

- Edite o exemplo com o IP/URL e o UIN corretos
- Copie-o para o aparelho como `player-config.json` em `/sdcard/smartsignage/`
- via ADB: o script `install-from-pc-adb` pode fazer isso
- ou copie pelo gestor de ficheiros, se o SO permitir escrever nessa pasta
- Abra (ou reinicie) o Player-AD

Se existir configuração **interna** da app e configuração no SD, a interna tem prioridade. Use a tela de configuração e **Aplicar e iniciar** para gravar de forma consistente.

5. Verificar se está a funcionar

Checklist rápido:

- **Rede** — o totem faz ping / abre o mesmo IP do servidor noutra browser da rede?
- **UIN** — é exatamente o código de ativação do totem no painel (maiúsculas/minúsculas e hífen contam)?
- **Painel web** — o totem aparece **online** / com heartbeat recente?
- **Playlist** — há mídias publicadas / aprovadas para esse totem?
- **Ecrã** — após **Aplicar e iniciar**, aparece reprodução (vídeo/imagem) e não fica só na tela de config?

Se o servidor estiver correto mas a playlist vazia, o player pode ficar à espera ou usar mídia de fallback (se existir).

6. Atualizar o Player-AD

- Obtenha o novo `Player-AD-release.apk`
- Instale por cima (pendrive ou ADB) — o Android pergunta **Atualizar**
- Em geral a configuração (`serverUrl` / `uin`) **mantém-se**
- Abra o app e confirme a reprodução

Se aparecer erro de assinatura (`INSTALL_FAILED_UPDATE_INCOMPATIBLE`):

- Desinstale a versão antiga
- Instale de novo
- **Reconfigure** `serverUrl`, `uin` e `deviceId` (a desinstalação pode apagar dados internos)

7. Problemas comuns

Sintoma	O que tentar
Pendrive não aparece	Outra porta USB; reinserir; formatar em FAT32/exFAT
Não instala o APK	Permitir apps desconhecidas; confirmar que o ficheiro é <code>Player-AD-release.apk</code>
App abre e fecha	Reiniciar o aparelho; reinstalar; pedir log ao suporte
Não liga ao servidor	Confirmar <code>serverUrl</code> (IP, porta :8080), firewall e Wi-Fi do totem
Totem offline no painel	UIN errado; rede; aguardar o intervalo de heartbeat
Imagem/vídeo deitado	Ajustar <code>displayRotation</code> / <code>screenOrientation</code> ; ver manual operacional
"Preencha serverUrl, uin e deviceId"	Completar os três campos e Aplicar e iniciar

Informações úteis para o suporte

Ao pedir ajuda, envie:

- Modelo do aparelho / TV box
- Versão do Player-AD (se visível)
- URL do servidor usada (sem palavras-passe)
- Código de ativação (UIN)
- Foto ou texto da mensagem de erro

8. Resumo em 1 minuto

- Crie o totem no painel e **copie o código de ativação**
- Instale `Player-AD-release.apk` (pendrive ou ADB)
- Abra o player → **5 toques** → configuração
- Preencha **URL do servidor**, **UIN** e **deviceId** (iguais ao código)
- **Aplicar e iniciar**
- Confirme no painel que o totem está online e a playlist a reproduzir

9. Onde está cada ficheiro

Item	Local
Manual do utilizador (este)	Player-AD/docs/MANUAL-USUARIO-INSTALACAO-COFIGURACAO.md
Kit pendrive	install-pendrive/
APK no kit	install-pendrive/apk/Player-AD-release.apk
Exemplo de config	install-pendrive/config/exemplo-player-config.json
Config no Android	/sdcard/smartsignage/player-config.json
Manual técnico TV box	Player-AD/docs/MANUAL-OPERACIONAL-TVBOX.md
LEIA-ME rápido do USB	install-pendrive/LEIA-ME.txt

3. Manual operacional (TV box / totem)

Fonte: MANUAL-OPERACIONAL-TVBOX.md

Player-AD — Manual operacional (TV box / totem)

Guia de instalação, provisionamento, boot customizado e manutenção em campo.

Manual do utilizador (instalação + configuração):
MANUAL-USUARIO-INSTALACAO-CONFIGURACAO.md

Versão do app referida: 1.33

Arquitetura (dev): MANUAL-ARQUITETURA-DESENVOLVIMENTO.md

Hardware / procurement: ../../docs/hardware/README.md

1. Pré-requisitos

Item	Detalhe
PC	Windows com PowerShell 5.1+
JDK	Temurin 17 (JAVA_HOME)
ADB	Android Platform Tools no PATH
Python	3.x + Pillow (boot logo/animation)
Dispositivo	Android 7+ (API 24), USB debug autorizado
Rede	Servidor Totem Digital acessível (serverUrl)

2. Instalação completa (TV box totem)

```
cd Player-AD\scripts
.\install-player-adb.ps1
```

O script:

- Compila assembleRelease
- Copia APK para install-pendrive/apk/
- adb install -r -d -g
- Envia player-config.json → /sdcard/smartsignage/
- Provisiona kiosk (portrait, launcher, immersive) — **padrão totem**

Flags úteis

Flag	Uso
-SkipBuild	Só instala APK já compilado
-NoKioskSetup	Celular / teste — sem alterar launcher/rotação SO
-OpenConfig	Abre tela de configuração após install
-NoLaunch	Não inicia MainActivity
-ConfigJson "C:\path\config.json"	Config customizada
-UserRotation 1	Portrait no SO (default totem)

Instalação em celular (teste, sem “prender” o aparelho)

```
.\install-player-adb.ps1 -SkipBuild -NoKioskSetup -OpenConfig
```

Use "kioskMode": "immersive" no JSON (não strong).

3. Configuração (player-config.json)

Local externo: /sdcard/smartsignage/player-config.json

Local interno: filesDir/player-config.json (prioridade se existir)

Exemplo mínimo:

```
{
  "serverUrl": "http://217.216.91.135:8080",
  "uin": "CODIGO-UIN",
  "deviceId": "CODIGO-UIN",
  "kioskMode": "strong",
  "displayRotation": 0,
  "screenOrientation": "portrait",
  "acceptImagesInPlaylist": true,
  "allowPlaybackAudio": true,
  "fallbackPropagandasPerVinheta": 3,
  "batimentoCardiaco": 120,
  "maxSecondsWithoutServerCheck": 600
}
```

Orientação (displayRotation)

Valor	Modo
0	Retrato (portrait)
1	Paisagem (landscape)
2	Retrato invertido
3	Paisagem invertida

No totem portrait Allwinner: user_rotation=1 no SO (aplicado pelo script de install).

Kiosk

Modo	Comportamento
strong	Fullscreen + lock task + bloqueio Home/Recentes (se SO permitir)
immersive	Só fullscreen imersivo

Na **tela de config** (debug) o kiosk fica sempre relaxado.

4. Abrir configuração no totem

- **5 toques rápidos** no botão OK / centro (D-pad ou toque na tela)
- Toast: “Mais N toque(s) no OK...”

Salvar config → **Aplicar e iniciar** (barra fixa no rodapé).

5. Boot customizado (duas fases)

Fase 1 — Bootlogo (Allwinner, antes do Android)

Logo estático na partição bootloader (boot logo .bmp 1280×720).

```
cd Player-AD\scripts
.\install-bootlogo.ps1
# ou com PNG custom:
.\install-bootlogo.ps1 -InputLogo "C:\caminho\logo.png"
adb reboot
```

Assets: install-pendrive/bootanimation/logo-totemdigital-boot.png, boot logo .bmp

Fase 2 — Bootanimation (animação Android)

Texto TotemDigital + linha amarela, portrait 1080×1920:

```
.\install-bootanimation.ps1 -Portrait
adb reboot
```

Regenerar ZIPs (Python):

```
python build-bootanimation.py
python build-bootanimation.py --width 1080 --height 1920 --output ..\..\install-pendrive\bootanimation\
```

Parâmetros em build-bootanimation.py: TEXT_ROTATION_DEG, TEXT_SCALE, ACCENT_RGB.

Requisito: root (su) + /system gravável.

6. Rotação automática da mídia (v1.34+)

Se vídeo/imagem estiver em orientação diferente da config (ex.: landscape num totem portrait):

- **Vídeo:** transformação no TextureView (ExoPlayer)

- **Imagem:** rotação do bitmap antes de exibir

Log esperado (player-ad-operations.log ou logcat tag Player-AD):

```
[DISPLAY] Correção orientação vídeo 1920x1080 → 90° (mount=0)
```

Se não girar: ver secção 8 (troubleshooting).

7. Comandos do dia a dia

```
# Diagnóstico
.\diagnose-android-box.ps1

# Só rotação SO
.\set-android-display-rotation.ps1 -Rotation 1

# Log operacional
adb exec-out run-as br.com.smartchannel.playerad cat files/player-ad-operations.log

# Reiniciar app
adb shell am force-stop br.com.smartchannel.playerad
adb shell am start -n br.com.smartchannel.playerad/.ui.MainActivity

# Versão instalada
adb shell dumpsys package br.com.smartchannel.playerad | findstr versionName
```

8. Troubleshooting

App não instala

- Conflito de assinatura: script desinstala e reinstala automaticamente
- INSTALL_FAILED: verificar adb devices → device

Portrait não aplica

```
adb shell settings get system user_rotation # esperado: 1
adb shell settings get system accelerometer_rotation # esperado: 0
.\set-android-display-rotation.ps1 -Rotation 1
```

SuperSU pede root a cada ~15 min

- SuperSU → Player-AD → **Permitir sempre**
- v1.31+ evita su quando user_rotation já está correto

Bootanimation não muda

- Verificar /system gravável: diagnóstico no script
- Backup em /sdcard/smartsignage/backup/

Bootlogo não muda (Allwinner)

- Partição: /dev/block/mmcblk0p2

- Montagem: /mnt/boot/logo (script cria se não existir)
- Formato: BMP 1280×720 RGB 24-bit

Mídia não gira (v1.34)

- Confirmar versão ≥ 1.34 (`adb shell dumpsys package br.com.smartchannel.playerad | findstr versionName`)
- Ver log DISPLAY — deve mostrar `raw=`, `eff=`, `meta=` e `total=` graus
- Vídeo landscape + config portrait → `total=90` ou `270`
- Se invertido: reportar combinação (config + tipo mídia) para ajuste de sentido

Sem rede / offline

Ordem de fallback do PlayerController:

- Plano online (dispatch)
- Último plano persistido (`last-dispatch-plan.json`)
- Arquivos locais propagandas/ e vinhetas/

9. Estrutura de pastas no dispositivo

Caminho	Conteúdo
/sdcard/smartsignage/player-config.json	Config externa
filesDir/player-config.json	Config interna (prioridade)
filesDir/player-ad-operations.log	Log operacional
AppDirs/propagandas/	Cache de mídias
AppDirs/vinhetas/	Vinhetas locais
/system/media/bootanimation.zip	Animação boot Android

10. Deploy servidor (atualizar backend/frontend)

O Player-AD na box **não** usa `install-smartsignage.sh` para updates web.

No servidor staging/produção:

```
cd ~/TotemDigital && git pull
./scripts/deploy-backfront-build.sh
```

11. Referência rápida de scripts

Script	Função
install-player-adb.ps1	Build + install + kiosk
install-bootanimation.ps1	Boot animation Android
install-bootlogo.ps1	Bootlogo Allwinner
build-bootanimation.py	Gera ZIP bootanimation
build-bootlogo.py	PNG → bootlogo.bmp
diagnose-android-box.ps1	Diagnóstico ADB
set-android-display-rotation.ps1	Rotação SO

Última atualização: jul/2026 — Player-AD v1.34

4. Notas técnicas de instalação (kit pendrive / ADB)

Fonte: MANUAL-INSTALACAO-PLAYER-AD.md

SmartSignage Player-AD — Notas técnicas de instalação

Manual do utilizador (instalação + configuração):

MANUAL-USUARIO-PLAYER-AD.md

Cópia canónica no repositório: Player-AD/docs/MANUAL-USUARIO-INSTALACAO-CONFIGURACAO.md

Versão do app de referência: 1.67 (versionCode=68)

Data: 2026-07-16

1) Objetivo

Este documento (técnico) complementa o manual do utilizador e centraliza:

- montar o pendrive de instalacao;
- instalar via ADB (PC + cabo USB/OTG);
- instalar diretamente com pendrive na TV Box;
- orientar equipe tecnica e utilizador final;
- reduzir erros de assinatura, Java/Gradle e configuracao.

2) Estrutura oficial do kit pendrive

Use a pasta install-pendrive como raiz do conteudo a copiar para o USB.

```
install-pendrive/  
  apk/  
    Player-AD-release.apk  
    COLOQUE_O_APK_AQUI.txt  
  config/  
    exemplo-player-config.json  
  midias/  
    README.txt  
    propagandas/      (opcional)  
    vinhetas/         (opcional)  
  scripts/  
    install-from-pc-adb.bat  
    install-from-pc-adb.sh  
    install-on-android.sh  
  docs/  
    MANUAL-INSTALACAO-PLAYER-AD.md  
    MANUAL-INSTALACAO-PLAYER-AD.pdf  
  README.md  
  LEIA-ME.txt
```

3) Pre-requisitos tecnicos

3.1 Para build no PC

- Windows com PowerShell.
- JDK 17 (Temurin recomendado) configurado.
- Gradle Wrapper do projeto (Player-AD/gradlew.bat).

3.2 Para instalacao via ADB

- Android Platform Tools (adb) no PATH.
- TV Box com:
 - Opcoes de programador ativas;
 - Depuracao USB ativa;
 - autorizacao RSA concedida ao PC.

3.3 Para instalacao por pendrive (sem PC)

- Gestor de ficheiros na TV Box.
- Permissao para instalar apps desconhecidas.

4) Como gerar e atualizar o kit no repositório

4.1 Build release do app

No projeto Player-AD:

```
cd C:\SmartSignage-Pro\Player-AD
.\gradlew.bat assembleRelease --no-daemon
```

APK esperado:

- Player-AD\build\outputs\apk\release\Player-AD-release.apk

4.2 Copiar APK para o kit pendrive

```
$apk = "C:\SmartSignage-Pro\Player-AD\build\outputs\apk\release\Player-AD-release.apk"
Copy-Item -Force $apk "C:\SmartSignage-Pro\install-pendrive\apk\Player-AD-release.apk"
```

4.3 Ajustar configuracao de exemplo

Edite:

- install-pendrive\config\exemplo-player-config.json

Campos principais:

- serverUrl
- uin
- deviceId
- fallbackPropagandasPerVinheta

Exemplo:

```
{
  "serverUrl": "http://217.216.91.135:8080",
  "uin": "tot001",
  "deviceId": "android-tv-tot001",
  "fallbackPropagandasPerVinheta": 3
}
```

5) Instalacao via ADB (procedimento tecnico recomendado)

5.1 Windows

Na pasta install-pendrive:

```
scripts\install-from-pc-adb.bat
```

Esse script:

- localiza APK em apk*.apk;
- executa `adb install -r -d -g`;
- envia `config\exemplo-player-config.json` para `/sdcard/smartsignage/player-config.json` se existir.

5.2 Linux/macOS/Git Bash

```
bash scripts/install-from-pc-adb.sh
```

Com envio explicito de config:

```
bash scripts/install-from-pc-adb.sh "" config/exemplo-player-config.json
```

5.3 Tratamento de erro de assinatura

Se ocorrer:

- `INSTALL_FAILED_UPDATE_INCOMPATIBLE`

Execute:

```
adb uninstall br.com.smartchannel.playerad
adb install -r -d -g caminho/do/apk
```

6) Instalacao direta por pendrive (sem PC)

- Inserir USB na TV Box.
- Abrir gestor de ficheiros.
- Ir para `apk/`.
- Selecionar `Player-AD-release.apk`.
- Confirmar instalacao.
- Se solicitado, permitir instalacao de apps desconhecidas.

Observacao:

- Este metodo nao depende de ADB.
- Ideal para operacao em campo quando nao ha notebook disponivel.

7) Script no Android (cenario root)

Arquivo:

- scripts/install-on-android.sh

Uso tipico:

```
adb push install-pendrive /sdcard/install-pendrive
adb shell su -c "sh /sdcard/install-pendrive/scripts/install-on-android.sh"
```

Sem root, preferir instalacao manual por gestor de ficheiros ou via ADB no PC.

8) Pos-instalacao e verificacao

8.1 Confirmar versao instalada

```
adb shell dumpsys package br.com.smartchannel.playerad | grep -E "versionCode|versionName"
```

8.2 Abrir app via ADB

```
adb shell am start -n br.com.smartchannel.playerad/.ui.MainActivity
```

8.3 Coletar log de erro

```
adb logcat -d | grep -i -E "playerad|AndroidRuntime|FATAL|Exception"
```

9) Localizacao de arquivos no Android

9.1 Configuracao

- /sdcard/smartsignage/player-config.json

9.2 Midias fallback

- /sdcard/Android/data/br.com.smartchannel.playerad/files/propagandas/
- /sdcard/Android/data/br.com.smartchannel.playerad/files/vinhetas/

10) Troubleshooting rapido

- **adb nao encontra dispositivo**
 - verifique cabo de dados;
 - confirme depuracao USB;

- confirme popup RSA no Android.

- **App nao instala**

- valide que APK e release valido;
- tente desinstalar versao anterior (conflito de assinatura).

- **App fecha ao abrir**

- colete logcat;
- validar se APK atual foi realmente instalado;
- confirmar versao em `dumpsys package`.

11) Manual do utilizador final (resumo)

Passo a passo simples:

- Ligue a TV Box.
- Insira o pendrive.
- Abra o gestor de ficheiros.
- Entre em `apk/`.
- Toque em `Player-AD-release.apk`.
- Toque em Instalar.
- Abra o app SmartSignage Player-AD.

Se aparecer bloqueio de seguranca, permitir "instalar apps desconhecidas" para o gestor de ficheiros.

12) Boas praticas operacionais

- Sempre incrementar `versionCode` a cada release.
- Manter `versionName` coerente (ex.: 1.2, 1.3).
- Testar em 1 dispositivo antes da distribuicao em massa.
- Manter este manual junto do kit no USB.
- Guardar historico de versoes instaladas por cliente/site.

5. Manual de arquitetura e desenvolvimento

Fonte: MANUAL-ARQUITETURA-DESENVOLVIMENTO.md

Player-AD — Manual de Arquitetura para Desenvolvimento

Documento técnico para onboarding e manutenção do Player-AD.

1. Visão geral

O Player-AD é um player Android TV orientado a:

- buscar plano de mídia no backend (heartbeat + dispatch);
- fazer cache local de mídias;
- reproduzir vídeos/imagens em loop;
- operar em modo offline com fallback local;
- expor tela de debug para configuração e diagnóstico.

2. Blocos estruturais

2.1 Bootstrapping e ciclo de vida

- `PlayerAdApplication`
 - inicializa logger (`PlayerAdLogger.init()`);
 - semeia fallback local a partir de assets (`FallbackSeeder.seedFromAssetsIfNeeded()`);
 - cria e inicializa `MediaCacheManager`.
- `BootCompletedReceiver`
 - recebe `ACTION_BOOT_COMPLETED`;
 - inicia `MainActivity` automaticamente.

2.2 UI e Orquestração

- `MainActivity`
 - tela principal em modo kiosk/imersivo;
 - cria `ExoPlayer` + `PlayerView` + `ImageView`;
 - inicia `PlayerController.start()`;
 - mantém watchdog interno para reiniciar o loop caso ele termine ou falhe;
 - abre `DebugConfigActivity` no primeiro arranque ou por gesto oculto (**5 toques** no OK/centro).
- `DebugConfigActivity`
 - edição de `serverUrl`, `uin`, `deviceId`;
 - testes de conectividade (heartbeat, dispatch);
 - exibe log operacional e estado offline;

- persiste `player-config.json` interno (e tenta externo).

2.3 Integração backend

- `DispatcherApiClient`
 - `getToken()`, `heartbeat()`, `getDispatchPlan(token)`;
 - retry controlado em 401 com renovação de token;
 - `resolveUrl()` para transformar URL relativa em absoluta.
- `PlayerEventsClient`
 - envia telemetria de playback em `/api/player/event`;
 - recupera token automaticamente (`heartbeat` / `getToken`) quando recebe 401.

2.4 Playback e fallback

- `PlayerController`
 - parseia `DispatchPlan`;
 - faz pré-cache assíncrono;
 - executa loop de playback;
 - aplica timeout por vídeo para avançar mídias travadas;
 - persiste último plano online e fonte do plano;
 - fallback na ordem:
 1. plano online;
 2. plano persistido em disco;
 3. fallback local propagandas/vinhetas.

2.5 Armazenamento local

- `AppDirs`
 - resolve raiz de dados (`externalFilesDir` com fallback para `filesDir`);
 - padroniza diretórios:
 - propagandas/
 - vinhetas/
- `MediaCacheManager`
 - mantém `metadata.json`;
 - controla validade de arquivo em cache;
 - aplica limpeza LRU (janela > 8 dias).
- Persistência operacional
 - log: `filesDir/player-ad-operations.log`;
 - último plano: `<AppDirs.root>/last-dispatch-plan.json`;
 - fonte atual do plano: `<AppDirs.root>/current-plan-source.txt`.

3. Fluxo principal de execução

- App inicia (`PlayerAdApplication.onCreate()`).
- `MainActivity.onCreate()`:
 - configura UI;
 - cria player;
 - decide entre abrir debug (first-run) ou iniciar player.
- `PlayerController.start()`:
 - heartbeat;
 - dispatch;
 - persiste plano;
 - pré-carrega mídias;
 - entra em loop.
- A cada ciclo completo da playlist:
 - tenta novo heartbeat + dispatch;
 - atualiza plano e cache quando disponível.
- Em falha de rede/dispatch:
 - tenta plano persistido;
 - senão, fallback local.

4. Classes e métodos principais

MainActivity

- `startPlayer()`
- carrega config via `PlayerConfigLoader`;
- instancia `DispatcherApiClient` e `PlayerController`;
- dispara coroutine para `playerController.start()`.
 - `openDebug(reason)`
- interrompe playback em execução;
- abre `DebugConfigActivity` com motivo.
 - `registerDevTap()`
- contador temporal para desbloquear debug por interação oculta.
 - `enterImmersiveMode()`
- aplica modo fullscreen/kiosk (`WindowInsets` + fallback legado).

PlayerController

- `start()`

- handshake inicial online + fallback inteligente.
 - `parseDispatchPlan(json)`
- converte JSON para `DispatchPlan` tipado.
 - `preloadPlan(plan)`
- limpa cache e baixa mídias ausentes.
 - `playLoop(plan, token, initialSource)`
- loop principal de reprodução com atualização periódica do plano.
 - `playItem(plan, item, token)`
- trata imagem/vídeo;
- envia eventos de início/fim;
- atualiza token retornado pelo `PlayerEventsClient`.
 - `buildFallbackPlan()`
- gera playlist local N:1 (propagandas:vinheta) via `fallbackPropagandasPerVinheta`.
 - `saveDispatchPlanToDisk()` / `loadDispatchPlanFromDisk()`
- persistência do último dispatch válido.
 - `updatePlanSource()`
- registra e persiste origem atual do plano (`ONLINE`, `PERSISTED`, `FALLBACK_LOCAL`).

`DispatcherApiClient`

- `getToken()`
- busca token em `/api/player/token`.
 - `heartbeat()`
- mantém sessão viva e atualiza token.
 - `getDispatchPlan(token)`
- obtém plano em `/api/player/dispatch`.
 - `resolveUrl(relativeOrAbsolute)`
- normaliza URLs para download/stream.

`PlayerEventsClient`

- `sendEvent(...)`
- envia eventos (`video_playback_start/end`, `image_display`, etc.);
- em 401, tenta renovação de token e reenvio.

`MediaCacheManager`

- `init()`
- carrega `metadata.json`.

- `onDownloadCompleted(...)`
- grava metadados de mídia baixada.
 - `onPlayFromCache(mediaId, today)`
- atualiza `lastAccessed` e contador diário `sum_play`.
 - `cleanupIfNeeded()`
- remove LRU de itens antigos até atingir limite.
 - `getMetadata(mediaId)`
- retorna estado tipado do cache.

DebugConfigActivity

- `runTestHeartbeat()`
- valida conectividade básica e token.
 - `runTestDispatchPlan()`
- valida retorno do plano e apresenta resumo.
 - `saveConfigInternal(cfg)`
- persiste config em arquivo interno.
 - `buildOfflineStateText()`
- mostra diagnóstico de plano persistido, fonte e contagem de fallback local.

5. Contratos de dados principais

5.1 Configuração (`player-config.json`)

Campos:

- `serverUrl` (obrigatório)
- `uin` (obrigatório)
- `deviceId` (obrigatório)
- `acceptImagesInPlaylist` (opcional, default `true`)
- `fallbackPropagandasPerVinheta` (opcional, mínimo 1, default 3)

Ordem de leitura (`PlayerConfigLoader`):

- `filesDir/player-config.json`
- `/sdcard/smartsignage/player-config.json`
- defaults embutidos.

5.2 Dispatch plan

Fonte: endpoint `/api/player/dispatch`.

Campos usados:

- `plan.playlistId`
- `plan.playlistName`
- `plan.campaignId`
- `plan.mediaItems[]` com:
 - `mediaId`
 - `url`
 - `duration`
 - `mediaType`

5.3 Eventos de telemetria

Endpoint: `/api/player/event`.

Estrutura inclui:

- `eventType`
- `mediaId / playlistId / campaignId`
- `duration`
- `completed`
- `metadata` (inclui `deviceId` e `player=Player-AD`).

6. Decisões técnicas relevantes

- **Fallback resiliente:** player não depende 100% da rede para iniciar/continuar.
- **Token resiliente:** backend 401 não interrompe playback; tenta renovação.
- **Imagens fora do ExoPlayer:** exibidas via `ImageView` para reduzir complexidade.
- **Persistência de estado operacional:** logs e fonte de plano ficam acessíveis no debug.
- **Compatibilidade Android TV/STB:** `AppDirs` evita dependência rígida de storage externo.
- **Orientação da mídia (v1.33):** `MediaViewportRotation` corrige vídeo (`TextureView` transform) e imagem (rotação bitmap) quando a orientação natural da mídia difere de `displayRotation` na config.
- **Boot custom (operacional):** scripts `build-bootanimation.py`, `build-bootlogo.py`, `install-bootanimation.ps1`, `install-bootlogo.ps1` — ver manual operacional.

2.6 Apresentação e orientação

- `DisplayPresentationController` — aplica rotação SO + fallback visual (`ViewDisplayRotation`).
- `PortraitViewportLayout / PortraitViewportMetrics` — viewport 9:16.
- `MediaViewportRotation` — correção por item de mídia no playback.
- `SystemDisplayRotation` — `user_rotation` via Settings ou su.

7. Pontos de atenção para evolução

- `MainActivity.currentVersionCode()` usa API legada (`versionCode` depreciado).

- `PlayerEventsClient.sendEvent()` ignora falhas de telemetria por design (não derruba playback).
- Limpeza LRU considera apenas mídias válidas sem acesso recente (> 8 dias); revisar estratégia para cenários de storage crítico.
- `DebugConfigActivity` hoje preserva `fallbackPropagandasPerVinheta` carregando valor atual; se virar campo editável na UI, ajustar `readConfigOrNull`.

8. Checklist para mudanças seguras

- Alterou contrato backend?
- revisar `DispatcherApiClient`, `PlayerEventsClient`, `parseDispatchPlan`.
- Alterou mídia/layout de playback?
- testar vídeo + imagem + fallback local.
- Alterou persistência?
- validar `last-dispatch-plan.json`, `current-plan-source.txt`, `metadata.json`.
- Alterou fluxo de boot/kiosk?
- validar `BootCompletedReceiver`, first-run debug, retorno de `DebugConfigActivity`.
- Alterou instalação/distribuição?
- atualizar scripts em `install-pendrive/scripts` e documentação associada.

9. Referências internas

- `Player-AD/docs/MANUAL-OPERACIONAL-TVBOX.md` — instalação, boot, config, troubleshooting (campo)
- `Player-AD/docs/REGISTRO-OPERACIONAL.md`
- `Player-AD/docs/TROUBLESHOOTING-CRASH.md`
- `docs/hardware/README.md` — índice hardware, ODM spec, SoC boot paths
- `docs/hardware/TOTEM-ODM-SPEC-v1.md`
- `install-pendrive/README.md`

6. Troubleshooting — crash

Fonte: TROUBLESHOOTING-CRASH.md

Player-AD – “aplicativo apresenta erro” / crash na TV

1) Garantir APK recente (layout Media3)

Se o `activity_main.xml` ainda usasse `com.google.android.exoplayer2.ui.PlayerView` com dependências **Media3**, o app pode fechar ao abrir. O layout correto é `androidx.media3.ui.PlayerView`.

Recompile e instale de novo:

```
cd ~/SmartSignage-Pro/Player-AD
./scripts/auto_install.sh
```

2) HTTP (LAN) com `targetSdk` alto

Com `targetSdk` 34, chamadas `http://` ao servidor podem ser bloqueadas sem `usesCleartextTraffic`. O manifest do Player-AD define `android:usesCleartextTraffic="true"` para uso em rede local.

3) Capturar o erro real (logcat)

Sem `rg`, use o script:

```
cd ~/SmartSignage-Pro/Player-AD
./scripts/capture-crash-log.sh
```

Ou manualmente:

```
adb logcat -c
adb shell am start -n br.com.smartchannel.playerad/.ui.MainActivity
# reproduza o erro
adb logcat -d | grep -iE "AndroidRuntime|FATAL|playerad|Player-AD"
adb logcat -b crash -d
```

Envie as linhas com **FATAL EXCEPTION** e **Caused by**.

4) Falha de rede / Dispatcher

Se não houver crash, mas tela preta: o heartbeat pode estar falhando (IP errado, firewall, HTTPS obrigatório). Veja `adb logcat` filtrando Player-AD após o build com `Log.e` em `MainActivity`.

7. Registro operacional

Fonte: REGISTRO-OPERACIONAL.md

Registo operacional do Player-AD

Onde fica o ficheiro

- **Nome:** `player-ad-operations.log`
- **Localização:** diretório privado do app — `getFilesDir()` do processo

Caminho típico em dispositivo:

`/data/data/br.com.smartchannel.playerad/files/player-ad-operations.log`

O ficheiro **não** fica acessível ao utilizador via gestor de ficheiros; só via:

- **Ecrã de debug** do próprio app (modo desenvolvimento → configuração): secção **Registo operacional**, com **Atualizar registo** e **Limpar registo**.
- **ADB** (debug USB), com o pacote instalado:

```
adb shell run-as br.com.smartchannel.playerad cat files/player-ad-operations.log  
# ou copiar para o PC:
```

```
adb exec-out run-as br.com.smartchannel.playerad cat files/player-ad-operations.log > player-ad-operati
```

O que é registado

- Linhas com **data/hora** (yyyy-MM-dd HH:mm:ss.SSS), nível [I|W|E], **categoria** e mensagem.
- **DISPATCH:** recebimento de plano (playlist id/nome, nº de itens, campaignId quando existir).
- **PLAYBACK:** início/fim de imagem ou vídeo.
- **FALLBACK / CACHE / MAIN / DEBUG_UI:** erros e ações relevantes.

Espelho em **Logcat** com tag `Player-AD`.

Limites

- Truncagem automática quando o ficheiro ultrapassa ~512 KB (mantém as últimas ~1500 linhas).

Ver no app

Abrir o ecrã de **configuração / debug** (toques secretos ou atalho de desenvolvimento) e descer até **Registo operacional**.